

第17-18课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

第 17-18课 PDF 解析整理版

第 17 课+第 18 课：从"分散的零件"走向"完整生产线"

- 第 17 课：多表 Join 路径自动推理
- 第 18 课：Schema Linking 完整链路整合与效果评估

[要点] 两课核心关系

第 17 课和第 18 课是 Schema Linking 子模块这条线从"分散的零件"走向"完整生产线"的关键两节课：

- 第 17 课（补齐最后一块拼图）：解决了 Schema Linking 三模块中的"连接器"问题。前 15、16 课解决了"选对表""选对字段"，但多表查询时 FROM 子句主表选错（锚表选错）会让 SQL 看似正确但语义错位；表之间怎么连也容易写漏或写错。17 课通过查询意图识别 + 图算法（BFS）自动推导，让 SQL 生成前先得到"对的主表 + 对的连接路径"。
- 第 18 课（Pipeline 整合与评估）：把 15、16、17 课三个独立模块串联为一个 schema_link() 入口函数，替换系统中硬编码的全量 Schema 注入。端到端评估显示 Token 节省 83-95%，同时 SQL 准确率持平或提升。

[链接] 逻辑衔接点

- 模块复用：18 课的 Pipeline 直接调用 17 课的 select_anchor(query, candidate_tables) 作为第 2 步，调用 17 课的 resolve_joins(anchor, target_tables) 作为第 4 步，17 课是 18 课的"连接器提供者"。
- 接口契约：18 课 Pipeline 入口函数 schema_link() 的输出字典结构（tables / fields / join_path / dynamic_schema）对应 15 课、16 课、17 课三个模块输出结果的并集，是一个上层抽象。
- 数据流闭环：14 课（手写向量检索）→ 15 课（LangChain + ChromaDB 框架化）→ 16 课（字段级语义匹配）→ 17 课（Join 路径推理）→ 18 课（Pipeline 整合）—— 18 课是这个子模块的"集成测试与上线评估"。
- Fallback 兼容：18 课在 prompt_builder.py 中新增 use_schema_linking 参数，向后兼容（不传 = 走原全量 Schema），是 9-10 课"服务化+向后兼容"思路的延续。

两节课的关系 / 总结

维度

第 17 课（多表 Join 路径自动推理）

第 18 课（Schema Linking 完整链路整合与效果评估）

所处阶段

第四阶段《让系统"理解业务"——SchemaLinking+RAG》第五节

第四阶段《让系统"理解业务"——SchemaLinking+RAG》第六节

核心问题

多表查询时锚表选错、Join 条件写错

三个独立模块没有统一入口，系统中仍是硬编码全量 Schema

解决思路

意图识别 + 表关系图 + BFS 最短路径算法

Pipeline 编排 + 动态 Schema 组装 + Fallback 兼容 + 端到端评估

关键产出

join_resolver.py (含 select_anchor + _bfs_shortest_path + _build_on_clause)

schema_linker.py (含 schema_link() 入口 + _assemble_dynamic_schema) + prompt_builder.py 新增 use_schema_linking 参数

在链路中的位置

解决"选对主表 + 写对 Join"——Schema Linking 第 1 步（锚表）+ 第 4 步（Join 路径）

把第 15-17 课的三个模块串联为可调用的端到端 Schema Linking 服务

衔接关系

第 17 课让 Schema Linking "能连对"，第 18 课让 Schema Linking "能上线"

第 18 课的 Pipeline 是 17 课模块的上游消费者 + 13 课"Schema Linking 问题"的最终答案

一句话关系总结：

- 第 17 课用图算法 + 意图识别解决了"表之间怎么连、谁当主表"的问题，让 SQL 生成有正确的脚手架；
- 第 18 课用 Pipeline 整合 + Fallback 兼容 + 端到端评估把 15、16、17

课三个独立模块"打包"为可调用的 `schema_link()` 入口，把"研究代码"变成"生产模块"。

- 18 课的 Pipeline 调用链: `query` → `table_retriever` (15课) → `select_anchor` (17课) → `field_matcher` (16课) → `resolve_joins` (17课) → `_assemble_dynamic_schema` → `dynamic_schema` → `prompt_builder` (注入)，第 17 课是这条链路中"锚表选择 + Join 路径推理"两个关键步骤的实现者。

第 17 课：多表 Join 路径自动推理

课程定位与目标

课程定位：

- 所处阶段：第四阶段《让系统"理解业务"——SchemaLinking+RAG》第五节
- 本课核心问题：第 14-16 课解决了"选对表"和"选对字段"，但表之间怎么连、哪张表放在 FROM 子句，仍然会让模型犯错；本课通过查询意图理解 + 图算法自动推导 Join 路径和条件。

本节课目标：

- 理解锚表选择在 SQL 生成中的重要性
- 能根据用户问题意图判断合理的锚表
- 掌握 BFS 最短路径算法在 Join 推导中的应用
- 实现 `join_resolver.py` 模块，支持两表 / 三表 / 多表的 Join 自动生成

注意：本课不使用 Embedding，Join 路径推导是纯图算法；锚表选择使用轻量规则做查询意图识别。二者与第 14-16 课的语义匹配形成互补——语义匹配解决"选什么"，意图识别解决"主语是谁"，图算法解决"怎么连"。

开场引入

回顾当前系统已经具备的能力：

[要点] 用户问题："按客户类型统计各产品线的收入，需要换算成人民币"

表召回（第 15 课）

- `sales_orders`
- `dim_customers`
- `dim_products`
- `exchange_rates`

字段匹配（第 16 课）

- `net_amount`
- `customer_type`
- `product_line`
- `rate_to_cny`

但 SQL 的 FROM 子句该写哪张表？这四张表又该如何 Join？

如果让模型自己推断 Join 关系，常见错误不止漏 Join 或 Join 条件写反，还包括更隐蔽的锚表选错：

```
-- 错误 1: 漏 Join (忘了关联 exchange_rates)
SELECT ... FROM sales_orders
JOIN dim_customers ON ...
JOIN dim_products ON ...
-- 没有汇率换算

-- 错误 2: Join 条件错误
JOIN exchange_rates ON sales_orders.currency = exchange_rates.rate_date
-- currency 和 rate_date 类型都对不上

-- 错误 3: 锚表选错
FROM dim_customers
JOIN sales_orders ON ...
-- 聚合维度变成了客户，语义中心变成了"客户"而不是"收入"
```

锚表选错比漏 Join 更隐蔽，因为生成的 SQL 往往能执行，只是结果不是用户想要的。

例如"列出所有客户"和"按客户类型统计收入"都涉及 sales_orders 和 dim_customers，但主表完全不同：

问题

正确锚表

原因

按客户类型统计收入

sales_orders

问题的主体是"收入"这个指标

哪些客户没有下过订单

dim_customers

问题的主体是"客户"这个实体

[要点] 锚表 = SQL 中 FROM 后面的第一张表，它必须对应用户问题的语义主体。

一、锚表选择：先理解用户在问什么

1.1 为什么"事实表优先"不够

在星型模型中，SQL 的惯用写法是 FROM 事实表 JOIN 维度表，所以很多实现会简单粗暴地优先选事实表。

但这只在问题是指标型时成立。如果问题是实体型，把事实表当主表会过滤掉没有事实记录的实体。

问题：列出所有客户

-- 错误写法

```
FROM sales_orders
JOIN dim_customers ON sales_orders.customer_id = dim_customers.customer_id
```

这种写法只返回下过订单的客户；没有订单的客户被 INNER JOIN 过滤掉了，与"列出所有客户"的语义不符。

问题：哪些客户没有下过订单

-- 错误写法

```
FROM sales_orders
JOIN dim_customers ON sales_orders.customer_id = dim_customers.customer_id
```

这种写法永远找不到"没有订单的客户"。正确写法应该以 dim_customers 为主表，再用 LEFT JOIN 关联订单：

```
FROM dim_customers
LEFT JOIN sales_orders ON dim_customers.customer_id = sales_orders.customer_id
WHERE sales_orders.order_id IS NULL
```

1.2 查询意图：指标型 vs 实体型

判断锚表之前，先判断用户问题的意图：

- 指标型问题：用户在问收入、费用、毛利、数量等可聚合的数值
- 实体型问题：用户在问客户、产品、订单、汇率等具体对象

问题

意图

正确锚表

按客户类型统计收入

指标型

sales_orders

列出所有客户

实体型

dim_customers

哪些客户没有下过订单

实体型

dim_customers

各产品线的毛利率

指标型

sales_orders

上个月的研究费用

指标型

finance_expenses

查询汇率历史

实体型

exchange_rates

1.3 轻量意图识别规则

对于当前这个 5 张表的模型，可以用一组轻量规则做意图识别：

- 指标信号词：统计、多少、总计、平均、占比、总和、额、量
- 实体信号词：列出、哪些、所有、没有、明细、每个
- 强指标词：收入、销售额、费用、毛利、利润、金额、数量

当问题里出现强指标词时，优先判为指标型；出现实体信号词时，优先判为实体型。

判断完意图后，再把问题中的词和每张表的业务关键词做匹配：

表

类型

关键词

sales_orders

事实表

收入、销售额、订单、销售、数量、金额、毛利、利润

finance_expenses

事实表

费用、研发、销售费用、管理费用、财务费用、期间费用

dim_customers

维度表

客户、客户类型、OEM、储能集成商、电网集团

dim_products

维度表

产品、产品线、型号、SKU

exchange_rates

参考表

汇率、币种、人民币、美元、欧元

1.4 锚表选择策略

```
def select_anchor(query, candidate_tables):
    intent = classify_intent(query) # 指标型 / 实体型 / 模糊型
    scores = score_tables_by_keywords(query) # 每张表的关键词匹配分

    if intent == "metric":
        # 在事实表中选分数最高的
        return max(fact_tables, key=lambda t: scores[t])
    elif intent == "entity":
        # 在维度/参考表中选分数最高的
        return max(dim_tables, key=lambda t: scores[t])
    else:
        # 意图模糊时，选子图中心性最高的表
        return most_central_table(candidate_tables)
```

选好锚表后，还要做一步连通性校验：如果锚表无法到达某个目标表，说明这组表本身不应该被 Join 在一起。

最典型的例子是 `sales_orders` 和 `finance_expenses`：两张都是事实表，业务上独立，无法连通。此时正确的处理不是硬连，而是提示“这些表需要独立查询”。

二、表关联图构建

2.1 概念模型

确定锚表之后，剩下的问题是：从锚表出发，如何把其他目标表连进来？

把数据库的表间关系看作一张有向图：

- 节点 = 表
- 边 = Join 关系，包含 Join 条件
- 边的方向 = 从一个表指向另一个表
- 边的属性 = 外键列、主键列、Join 类型

注意：Join 类型与方向有关。

- 从事实表到维度表用 JOIN：每条事实记录都有对应的维度值
- 从维度表到事实表用 LEFT JOIN：不是所有维度值都有事实记录

这种方向性直接影响“哪些客户没有订单”这类存在性查询的正确性。

2.2 数据结构设计

用 Python 字典表示邻接表：

```
TABLE_RELATIONSHIPS = {
    "sales_orders": [
        {
            "target": "dim_customers",
            "fk_col": "customer_id",
            "pk_col": "customer_id",
            "join_type": "JOIN",
        },
        {
            "target": "dim_products",
            "fk_col": "product_id",
            "pk_col": "product_id",
            "join_type": "JOIN",
        },
        {
            "target": "exchange_rates",
            "fk_col": "order_date, currency",
            "pk_col": "rate_date, currency",
            "join_type": "LEFT JOIN",
        },
    ],
    "dim_customers": [
        {
            "target": "sales_orders",
            "fk_col": "customer_id",
            "pk_col": "customer_id",
            "join_type": "LEFT JOIN",
        },
    ],
    ...
}
```

2.3 为什么用手动配置而非自动从 information_schema 提取

方式

优势

劣势

手动 JSON 配置

精确控制、可标注 Join 类型、支持复合键、可配置优先级

新增表需手动维护

自动从 FK 提取

零维护成本

很多业务关系不建外键、无法区分 JOIN/LEFT JOIN、不支持复合键

本课程选择手动配置，因为：

1. 企业数据库经常不建外键约束，只在数据层面保持关系
2. 需要标注 Join 类型，例如从维度表到事实表必须 LEFT JOIN
3. 支持复合键，例如 order_date + currency 关联 rate_date + currency

2.4 独立表的处理

在当前模型中 finance_expenses 与 sales_orders 没有直接关联关系。这不是 Bug——费用数据是独立的期间汇总，不能和订单明细直接 Join。

当系统检测到目标表中包含无法连通的表时，应该：

1. 标记为"无法连通"
2. 建议独立查询
3. 或提示用户这可能是两个不同的分析问题

三、BFS 最短路径 Join 推导

3.1 算法概述

已知锚表和目标表集合后，Join 推导分三步：

1. 选锚表：根据查询意图确定 FROM 子句主表

2. 找路径：从锚表出发，用 BFS 找到达每个目标表的最短路径
3. 合并去重：将所有路径合并，去除重复的 Join 边

3.2 BFS 最短路径

标准 BFS：

```
from collections import deque

def _bfs_shortest_path(start, end, relationships):
    if start == end:
        return [start]

    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        current, path = queue.popleft()
        for edge in relationships.get(current, []):
            next_table = edge["target"]
            if next_table in visited:
                continue
            new_path = path + [next_table]
            if next_table == end:
                return new_path
            visited.add(next_table)
            queue.append((next_table, new_path))

    return None # 不可达
```

BFS 保证找到的是最短路径（经过的表最少），对应最少的 Join 数量。

3.3 合并路径生成 Join 序列

```
for target in target_tables:
    if target == anchor:
        continue
    path = _bfs_shortest_path(anchor, target)
    if path is None:
        unreachable.append(target)
        continue

    for i in range(len(path) - 1):
        from_t, to_t = path[i], path[i + 1]
        edge_key = (from_t, to_t)
        if edge_key in visited_edges:
            continue # 去重：同一条边不重复 Join
        visited_edges.add(edge_key)
        join_info = get_join_info(from_t, to_t)
        joins.append(join_info)
```

[要点] 关键：visited_edges 去重机制。当多个目标表的路径共享中间节点时，不会产生重复 Join。

3.4 复合键 Join

需要通过两个字段同时匹配 exchange_rates:

```
{
    "target": "exchange_rates",
    "fk_col": "order_date, currency",
    "pk_col": "rate_date, currency",
    "join_type": "LEFT JOIN",
}
```

生成的 ON 子句:

```
sales_orders.order_date = exchange_rates.rate_date
AND sales_orders.currency = exchange_rates.currency
```

处理逻辑:

```
def _build_on_clause(from_table, to_table, edge):
    fk_cols = [c.strip() for c in edge["fk_col"].split(",")]
    pk_cols = [c.strip() for c in edge["pk_col"].split(",")]
    conditions = []
    for fk, pk in zip(fk_cols, pk_cols):
        conditions.append(f"{from_table}.{fk} = {to_table}.{pk}")
    return " AND ".join(conditions)
```

四、完整场景验证

场景 1：指标型两表 Join

问题：按客户类型统计收入

候选表：[sales_orders, dim_customers]

意图：指标型

锚表：sales_orders

路径：sales_orders → dim_customers

生成 SQL:

```
sales_orders
JOIN dim_customers ON sales_orders.customer_id = dim_customers.customer_id
```

场景 2：实体型查询

问题：列出所有客户

候选表：[dim_customers]

意图：实体型

锚表：dim_customers

生成 SQL:

```
dim_customers
```

场景 3：存在性查询（LEFT JOIN 的关键作用）

问题：哪些客户没有下过订单

候选表：[dim_customers, sales_orders]

意图：实体型

锚表：dim_customers

路径：dim_customers → sales_orders

生成 SQL：

```
dim_customers
LEFT JOIN sales_orders ON dim_customers.customer_id = sales_orders.customer_id
```

注意这里使用了 LEFT JOIN，因为从维度表到事实表不是所有维度值都有事实记录。

场景 4：三表 Join

问题：各产品线的毛利率

候选表：[sales_orders, dim_products]

意图：指标型

锚表：sales_orders

路径：sales_orders → dim_products

生成 SQL：

```
sales_orders
JOIN dim_products ON sales_orders.product_id = dim_products.product_id
```

场景 5：四表 Join + 复合键（最大复杂度）

问题：按客户类型统计各产品线的收入，需要换算成人民币

候选表：[sales_orders, dim_customers, dim_products, exchange_rates]

意图：指标型

锚表：sales_orders

生成 SQL：

```
sales_orders
JOIN dim_customers ON sales_orders.customer_id = dim_customers.customer_id
JOIN dim_products ON sales_orders.product_id = dim_products.product_id
LEFT JOIN exchange_rates ON sales_orders.order_date = exchange_rates.rate_date
AND sales_orders.currency = exchange_rates.currency
```

场景 6：独立表不可连通

问题：销售收入和期间费用对比

候选表：[sales_orders, finance_expenses]

锚表：sales_orders

BFS 结果：finance_expenses 不可达

输出：

[!] 无法连通的表: ['finance_expenses']
 → 这些表需要独立查询, 不能通过 Join 关联

这是预期行为。销售收入和期间费用是不同粒度、不同业务含义的数据, 强行 Join 没有业务意义。

五、运行演示

```
uv run join_resolver.py
```

输出（节选）：

```
=====
多表 Join 路径自动推理演示
=====
```

问题：按客户类型统计收入

候选表: ['sales_orders', 'dim_customers']

锚表: sales_orders (问题询问指标, 在事实表中选择最相关的 'sales_orders' 作为锚表)

生成 SQL 片段:

```
sales_orders
```

```
JOIN dim_customers ON sales_orders.customer_id = dim_customers.customer_id
```

问题：哪些客户没有下过订单

候选表: ['dim_customers', 'sales_orders']

锚表: dim_customers (问题询问实体, 在维度/参考表中选择最相关的 'dim_customers' 作为锚表)

生成 SQL 片段:

```
dim_customers
```

```
LEFT JOIN sales_orders ON dim_customers.customer_id = sales_orders.customer_id
```

问题：按客户类型统计各产品线的收入, 需要换算成人民币

候选表: ['sales_orders', 'dim_customers', 'dim_products', 'exchange_rates']

锚表: sales_orders (问题询问指标, 在事实表中选择最相关的 'sales_orders' 作为锚表)

生成 SQL 片段:

```
sales_orders
```

```
JOIN dim_customers ON sales_orders.customer_id = dim_customers.customer_id
```

```
JOIN dim_products ON sales_orders.product_id = dim_products.product_id
```

```
LEFT JOIN exchange_rates ON sales_orders.order_date = exchange_rates.rate_date AND sales_orders.currency = exchange_rates.currency
```

问题：销售收入和期间费用对比

候选表: ['sales_orders', 'finance_expenses']

锚表: sales_orders (问题询问指标, 在事实表中选择最相关的 'sales_orders' 作为锚表)

[!] 无法连通的表: ['finance_expenses']

这些表需要独立查询, 不能通过 Join 关联

六、与表召回、字段匹配的关系

三个模块各司其职, 串联后形成完整的 Schema Linking:

- table_retriever (第 15 课): 输入用户问题 → 输出相关表列表
- select_anchor (本课): 输入用户问题 + 候选表 → 输出锚表

- field_matcher (第 16 课): 输入用户问题 + 候选表 → 输出相关字段列表
- join_resolver (本课): 输入锚表 + 候选表 → 输出 Join 路径和 SQL 片段
- schema_linker (第 18 课): 将三者串联, 输出完整的动态 Schema

小结

本节课实现了 Schema Linking 的第三步——Join 路径推理, 并补上了关键的第一步: 锚表选择。

下一课将把表召回、字段匹配、Join 推理三个模块整合为完整的 Schema Linking Pipeline。

附录: 完整代码

完整代码文件路径:

```
code/chatbi_mvp/join_resolver.py
```

第 18 课: Schema Linking 完整链路整合与效果评估

课程定位与目标

课程定位:

- 所处阶段: 第四阶段《让系统"理解业务"——SchemaLinking+RAG》第六节
- 本课核心问题: 前三课分别实现了表召回、字段匹配、Join 推理三个独立模块。本课将它们串联为完整的 Schema Linking Pipeline, 替换系统中的硬编码全量 Schema 注入, 并做效果评估。

本节课目标:

- 实现 schema_linker.py 作为三模块的 Pipeline 编排入口
- 改造 prompt_builder.py, 支持动态 Schema 注入 (保留 fallback)
- 端到端对比"全量 Schema" vs "SchemaLinking 精简 Schema"的效果差异

开场引入

回顾第 15-17 课的三个模块:

```
table_retriever → 输入问题, 输出相关表列表
field_matcher → 输入问题+候选表, 输出相关字段列表
join_resolver → 输入锚表+候选表列表, 输出 Join 路径和 SQL 片段
                → 其中锚表由 select_anchor (问题, 候选表) 推导
```

它们各自能独立运行、独立测试。但要真正替换系统中的硬编码

Schema，需要一个编排层——把三者串联起来，把结果组装成 Prompt 可用的格式。

这就是本课要实现的 `schema_linker.py`。

一、Pipeline 架构设计

整体流程

模块间接口约定

步骤

输入

输出

模块

1

用户问题 (str)

相关表列表 (list[dict])

`table_retriever`

2

用户问题 + 候选表名列表

锚表 (str)

`join_resolver.select_anchor`

3

用户问题 + 候选表名列表

相关字段列表 (list[dict])

`field_matcher`

4

锚表 + 候选表名列表

Join 路径 (dict)

join_resolver.resolve_joins

5

表列表 + 字段列表 + Join 路径

动态 Schema 文本 (str)

schema_linker 内部

核心函数签名

```
def schema_link(
    query: str,
    table_top_k: int = 3,
    field_top_k: int = 12,
    table_threshold: float = 0.3,
    field_threshold: float = 0.25,
    include_join: bool = True,
) -> dict:
    """
    Schema Linking 完整 Pipeline

    Returns:
    {
        "tables": [...],
        "fields": [...],
        "join_path": {...},
        "dynamic_schema": str, # ← 这是最终产出, 注入 Prompt
        "metadata": {...},
    }
    """
```

二、动态 Schema 组装

输出格式设计

动态 Schema 需要在"信息完整"和"Token 节省"之间取平衡:

表: sales_orders (事实表)

- net_amount 不含税收入 (财务口径的销售额) ★强制包含
- region 订单销售区域 (冗余字段)
- order_date 订单日期, DATE 类型
- order_status 订单状态 ★强制包含

表: dim_customers (维度表)

- region 客户所属的销售大区

表间关联:

```
JOIN dim_customers ON sales_orders.customer_id = dim_customers.customer_id
```

设计要点：

1. 按表分组：清晰标注每张表的角色（事实表 / 维度表）
2. 字段精简描述：只取第一句话，避免冗长
3. 规则标注：被业务规则强制包含的字段标 ★，帮助模型注意
4. Join 条件明确：直接给出 ON 子句，模型不需要自己推断
5. 无字段匹配时：标注"参考全部字段"，提示模型该表所有字段都可能用到

组装逻辑

```
def _assemble_dynamic_schema(tables, fields, join_path):
    # 1. 按表分组字段
    table_fields_map = {t["table_name"]: [] for t in tables}
    for f in fields:
        if f["table"] in table_fields_map:
            table_fields_map[f["table"]].append(f)

    # 2. 生成各表的精简 Schema
    for table_name, field_list in table_fields_map.items():
        lines.append(f"表: {table_name} ({domain}) ")
        for f in field_list:
            short_desc = f["description"].split("。")[0]
            lines.append(f" - {f['field']} {short_desc}")

    # 3. 生成 Join 关系说明
    if join_path.get("joins"):
        for j in join_path["joins"]:
            lines.append(f" {j['join_type']} {j['to_table']} ON {j['on_clause']}")

    return "\n".join(lines)
```

三、改造 prompt_builder.py

改造策略

核心原则：向后兼容 + 渐进式替换

不直接删除全量 SCHEMA 常量，而是通过新参数 use_schema_linking 控制行为：

```
def build_prompt(
    user_question: str,
    use_few_shot: bool = True,
    use_rules: bool = False,
    use_guards: bool = False,
    indicator_knowledge: str = "",
    use_schema_linking: bool = False, # ← 第 18 课新增
) -> tuple[str, str]:
```

Fallback 机制

```
# 第 18 课新增：动态 Schema Linking
schema_text = SCHEMA # 默认使用全量 Schema
if use_schema_linking:
    try:
        from schema_linker import build_dynamic_prompt_schema
        dynamic_schema = build_dynamic_prompt_schema(user_question)
        if dynamic_schema:
            schema_text = dynamic_schema
    except Exception:
        pass # 任何异常都回退到全量 SCHEMA
```

Fallback 触发条件：

- 1. schema_linker 模块导入失败（索引未构建等）
 - 2. 向量检索无结果（table_count=0）
 - 3. 任何未预期的异常
- [要点] Fallback 设计是生产系统的关键。Schema Linking 是"锦上添花"——成功了能大幅减少Token、提高精度；失败了系统仍能用全量 Schema 正常工作。绝不能因为 Schema Linking 的 Bug 导致整个系统不可用。

使用方式对比

```
# 旧方式（全量 Schema，无 Schema Linking）
system_msg, prompt = build_prompt(question, use_rules=True, use_guards=True)

# 新方式（开启 Schema Linking）
system_msg, prompt = build_prompt(
    question, use_rules=True, use_guards=True, use_schema_linking=True
)
```

对调用方来说，只是多了一个参数。

四、端到端效果评估

评估维度

维度

说明

期望

Token 消耗

Prompt 中 Schema 部分的 token 数

降低 70%+

SQL 准确率

生成的 SQL 是否可执行且结果正确

持平或提升

响应延迟

增加向量检索后的总延迟

增加 < 200ms

Token 消耗对比

问题类型

全量 Schema

Schema Linking

节省比例

单表简单查询

~1500 tokens

~80 tokens

95%

两表 Join

~1500 tokens

~150 tokens

90%

三表复杂查询

~1500 tokens

~250 tokens

83%

当前 5 张表的场景节省已经很可观；如果表数扩展到 50 张（全量 ~15000 tokens），Schema Linking 的价值将更加突出。

SQL 准确率分析

Schema Linking 对准确率的影响是双面的：

正面影响（多数情况）：

- 模型注意力不被无关字段分散
- 业务规则强制约束避免了 gross_amount/net_amount 混淆
- Join 条件直接给出，不需要模型推断

风险（少数情况）：

- 表召回遗漏：某张必要的表没被召回 → SQL 缺少 Join
- 字段匹配遗漏：某个必要字段没被匹配 → SQL 中字段缺失
- 阈值设置不当：过高导致漏召回，过低导致噪声

缓解策略

风险

缓解措施

表召回遗漏

降低 table_threshold、增大 table_top_k

字段匹配遗漏

业务规则白名单兜底

Schema Linking 完全失败

Fallback 到全量 Schema

阈值不确定

可以 A/B 测试逐步调优

五、仍然存在的问题

问题类型

SchemaLinking 解决了"选对数据源"的问题，但以下问题仍未解决：

代码块问题："各产品线的毛利率"

在引入第 17 课锚表选择后，系统会把锚表定为 sales_orders（订单事实表），并与 dim_products 建立 Join，拿到 material_cost、labor_cost 等成本字段。

但模型如何知道"毛利率"的计算公式？

毛利率 = $(\text{net_amount} - (\text{material_cost} + \text{labor_cost}) * \text{quantity}) / \text{net_amount} * 100$

这个公式不在 Schema 里，不在字段描述里，也不在 Join 条件里。

它是"业务知识"——需要从指标知识库中检索。

第 19 课预告

下一课将实现指标知识库 + RAG 检索：

- 把"毛利率""费用率""客单价"等指标的计算公式存入向量数据库
- 用户问问题时，除了 SchemaLinking 还检索相关指标定义
- 把指标公式也注入 Prompt，让模型"知道怎么算"

小结

本节课完成了 Schema Linking 的完整链路整合：

关键成果：

- schema_linker.py: Pipeline 编排入口，一个函数搞定全链路
- 改造 prompt_builder.py: 新增 use_schema_linking 参数，向后兼容
- Token 节省 83-95%，同时保持甚至提升 SQL 准确率
- Fallback 机制确保系统稳定性

附录：完整代码

完整代码文件路径：

code/chatbi_mvp/schema_linker.py

附件（飞书无下载权限）：第 17 课：多表 Join 路径自动推理.pdf

附件（飞书无下载权限）：第 18 课：Schema Linking 完整链路整合与效果评估.pdf